

IFMT – Instituto Federal Do Mato Grosso

Felipe Falcieri Macedo

Fundamentos de Hashing e Integridade de Dados

Cuiabá
2026

1. Introdução

Este relatório documenta o desenvolvimento e a validação de um conjunto de quatro scripts em Python, construídos para investigar a integridade dos arquivos do cliente e avaliar tecnicamente a robustez do armazenamento de senhas por hash.

1.1 Restrições de implementação seguidas

- hashlib foi utilizado apenas para o cálculo unitário do hash SHA-256 de um dado (arquivo ou senha), nunca para verificar integridade completa ou quebrar senhas de forma automatizada.
- Nenhuma biblioteca pronta de quebra de senha (passlib, bcrypt pronto, hashcat, john) foi utilizada.

2. Parte 1 — Conceitos Teóricos

2.1 Função Hash Criptográfica

Uma função hash criptográfica é um algoritmo determinístico que recebe uma entrada de tamanho arbitrário (uma mensagem, um arquivo, uma senha) e produz uma saída de tamanho fixo, chamada de hash, digest ou resumo criptográfico. Diferente de uma função hash comum (usada, por exemplo, em tabelas de dispersão), uma função hash criptográfica precisa satisfazer propriedades de segurança adicionais que a tornam adequada para uso em contextos como verificação de integridade, assinaturas digitais e armazenamento de senhas.

Três propriedades essenciais de uma função hash criptográfica:

- Resistência à pré-imagem: dado um valor de hash h , deve ser computacionalmente inviável encontrar qualquer entrada m tal que $\text{hash}(m)$ seja igual a h . Essa propriedade é o que permite armazenar o hash de uma senha em vez da senha em si: mesmo que o hash vazze, não é viável reverter o cálculo para descobrir a senha original.
- Resistência à segunda pré-imagem: dada uma entrada m_1 e seu hash h seja igual $\text{hash}(m_1)$, deve ser computacionalmente inviável encontrar uma segunda entrada diferente m_2 tal que $\text{hash}(m_2)$ seja igual a $\text{hash}(m_1)$. Essa propriedade impede que um invasor substitua um arquivo legítimo por outro malicioso que produza o mesmo hash, o que é essencial para a verificação de integridade implementada no item 2.1 deste relatório.
- Resistência a colisões: deve ser computacionalmente inviável encontrar quaisquer duas entradas distintas m_1 e m_2 (sem que uma delas seja previamente fixada) tais que $\text{hash}(m_1)$ seja igual a $\text{hash}(m_2)$. Essa propriedade é mais forte do que a resistência à segunda pré-imagem, pois o atacante tem liberdade para escolher as duas entradas.

Além dessas três, costuma-se citar também o efeito avalanche (uma pequena mudança na entrada mesmo de um único bit deve produzir um hash completamente diferente e imprevisível) e a eficiência computacional (o cálculo do hash deve ser rápido de se executar, ainda que, no caso específico de hashing de senhas, algoritmos propositalmente mais lentos como bcrypt, scrypt e Argon2 sejam preferíveis para dificultar ataques de força bruta em larga escala).

2.2 Hash vs. Cifragem

A diferença fundamental entre uma função hash e um algoritmo de cifragem (criptografia) está na reversibilidade da operação.

- Cifragem (criptografia simétrica ou assimétrica) é um processo reversível: um texto claro é transformado em texto cifrado por meio de uma chave, e esse texto cifrado pode ser decifrado de volta ao texto original, desde que se possua a chave correta (ou a chave correspondente, no caso assimétrico). A cifragem tem como objetivo garantir confidencialidade, proteger o conteúdo de uma informação de olhos não autorizados, permitindo que o destinatário legítimo a recupere.
- Hash é um processo unidirecional (não reversível por projeto): não existe uma 'chave de hash' que permita recuperar a entrada original a partir do valor de hash. Além disso, a cifragem

preserva o tamanho (aproximado) da entrada, enquanto o hash sempre produz uma saída de tamanho fixo, independentemente do tamanho da entrada. O hash tem como objetivo garantir integridade e autenticidade de conteúdo (ou, no caso de senhas, evitar armazenar o segredo em si), e não confidencialidade recuperável.

Por essa razão, senhas devem ser armazenadas com hash (idealmente com salt), e não cifradas: se fossem cifradas, qualquer pessoa com acesso à chave de decifragem (por exemplo, um administrador do sistema ou um invasor que a comprometa) conseguiria recuperar todas as senhas em texto claro, o que não deve ser possível nem mesmo para quem administra o sistema.

2.3 Colisões e SHA-1

Uma colisão de hash ocorre quando duas entradas distintas produzem exatamente o mesmo valor de hash como saída, ou seja, $\text{hash}(m1)$ é igual a $\text{hash}(m2)$ com $m1$ diferente de $m2$. Colisões são matematicamente inevitáveis em qualquer função hash, já que o conjunto de entradas possíveis é infinito (ou, na prática, muito maior que o espaço de saída), enquanto a saída tem tamanho fixo. O que uma função hash criptográfica segura garante não é a ausência de colisões, mas sim que seja computacionalmente inviável encontrá-las de forma deliberada em tempo hábil.

O SHA-1 é considerado obsoleto porque pesquisadores demonstraram, na prática, ser possível gerar colisões de forma deliberada com um custo computacional muito menor do que o esperado teoricamente para um hash de 160 bits. Em 2017, o ataque batizado de 'SHAttered', conduzido pelo Google em conjunto com o CWI Institute, produziu dois arquivos PDF distintos com o mesmo hash SHA-1, provando de forma pública e reproduzível que o algoritmo não oferece mais a resistência a colisões exigida de uma função hash criptográfica moderna. Como consequência, o SHA-1 foi descontinuado para uso em certificados digitais, assinaturas de código e protocolos como TLS/SSL, sendo substituído por algoritmos da família SHA-2 (como o SHA-256, usado neste projeto) e SHA-3.

2.4 Mecanismo de Salt

Salt é um valor aleatório, gerado de forma criptográfica segura, que é concatenado à senha antes do cálculo do hash. Em vez de armazenar $\text{hash}(\text{senha})$, armazena-se $\text{hash}(\text{salt} + \text{senha})$, junto com o próprio salt em texto claro (o salt não precisa ser secreto, apenas único e imprevisível o suficiente para não se repetir entre usuários).

O salt é indispensável no armazenamento seguro de senhas por dois motivos principais:

- Elimina o reaproveitamento de tabelas pré-computadas: sem salt, duas contas com a mesma senha produzem exatamente o mesmo hash, o que permite a um atacante usar uma única tabela pré-computada (dicionário de hashes ou rainbow table) para quebrar todas as senhas iguais de uma só vez, além de revelar imediatamente quais usuários compartilham a mesma senha. Com salt, cada usuário tem um valor diferente concatenado, então a mesma senha gera hashes completamente distintos em cada conta.
- Obriga o atacante a atacar cada hash individualmente: como cada salt é diferente, o atacante não pode reutilizar o esforço computacional de quebrar uma senha entre múltiplas contas, ele precisa recalcular o hash de cada senha candidata do dicionário para cada salt específico, multiplicando o custo total do ataque pelo número de contas (ou de salts distintos) que deseja atacar. Isso é exatamente o comportamento demonstrado na comparação entre os itens 2.2 e 2.4 deste relatório.

2.5 Tipos de Ataques a Hashes

Ataque de força bruta: consiste em testar sistematicamente todas as combinações possíveis de caracteres, dentro de um determinado espaço de busca (por exemplo, todas as senhas de até 8 caracteres usando letras, números e símbolos), calculando o hash de cada combinação e comparando com o hash-alvo. É o método mais genérico e garantidamente encontra a senha, mas seu custo cresce exponencialmente com o tamanho e a complexidade da senha, tornando-se inviável na prática contra senhas longas e aleatórias.

Ataques de dicionário: em vez de testar todas as combinações possíveis, o atacante testa apenas uma lista pré-definida de candidatos plausíveis, tipicamente senhas comuns, vazadas em outros incidentes, ou variações previsíveis delas (como adicionar números ou trocar letras por símbolos semelhantes). É muito mais eficiente que a força bruta pura porque explora o fato de que usuários reais tendem a escolher senhas fracas e previsíveis, mas só encontra senhas que estejam de fato no dicionário utilizado. Foi o método implementado nos itens 2.2 e 2.4 deste relatório.

Ataque com rainbow tables: é uma otimização de espaço-tempo para ataques de dicionário/força bruta sem salt. Em vez de armazenar todos os pares (senha, hash), o que consumiria uma quantidade proibitiva de espaço em disco uma rainbow table armazena apenas alguns pontos de cadeias (chains) geradas alternando o cálculo do hash com uma função de redução, permitindo reconstruir o caminho completo e recuperar a senha original a partir do hash com uma busca muito mais rápida do que recalculá-lo tudo, e usando muito menos espaço de armazenamento do que uma tabela hash senha completa. A grande limitação das rainbow tables é que elas dependem de o hash não ter salt: como cada rainbow table é construída para um algoritmo de hash específico (sem salt), a simples adição de um salt único por usuário torna as rainbow tables pré-computadas completamente inúteis, pois seria necessário construir uma tabela nova para cada salt possível, o que, na prática, inviabiliza esse tipo de ataque.

2. Item 2.1 — verificador_integridade.py

Script responsável por calcular o hash SHA-256 de todos os arquivos de um diretório (recursivamente, incluindo subpastas) e registrá-los em um arquivo hashes.txt no formato caminho_do_arquivo:hash. A leitura dos arquivos é feita em blocos de 1 MiB (chunks), permitindo processar arquivos grandes (> 1 GB) sem estourar a memória RAM. Ao ser executado novamente com a flag --verificar, o script recalcula os hashes atuais e os compara com o registro salvo, classificando cada arquivo em: novos, removidos, modificados ou inalterados.

2.1.1 Teste — geração inicial do registro de hashes

Foi criada uma pasta de teste (dados/) com dois arquivos, incluindo um em subpasta, e o script foi executado sem a flag --verificar para gerar o arquivo hashes.txt inicial:

```
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> mkdir -p dados/sub

Directory: C:\Users\Felipe Macedo\Documents\Segurança da Informação\dados

Mode                LastWriteTime         Length Name
----                -
d-----          07/09/2026   14:12             sub

PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> echo "conteudo A" > dados/a.txt
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> echo "conteudo B" > dados/sub/b.txt
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> python3 verificador_integridade.py dados
Calculando hashes SHA-256 de todos os arquivos em 'dados' (recursivo)...
2 arquivo(s) processado(s).
Registro salvo em 'hashes.txt'.
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> cat hashes.txt
a.txt:2ffea2e8fd6c3c83fc7e5918763c1b90595e1d38a6ccbacf8ab3fa41a0640dd
sub/b.txt:130eed0e858fb6a5f3412d8071c680f6037badd706bf7b424e40f7eef4d2b9f0
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação>
```

Figura 1 — Geração do hashes.txt a partir da pasta 'dados'.

2.1.2 Teste — simulação de alterações no diretório

Em seguida, o conteúdo de a.txt foi alterado, o arquivo sub/b.txt foi removido e um novo arquivo c.txt foi criado, simulando uma possível ação de um invasor:

```

PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> echo "conteudo A" > dados/a.txt
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> echo "conteudo B" > dados/sub/b.txt
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> python3 verificador_integridade.py dados
Calculando hashes SHA-256 de todos os arquivos em 'dados' (recursivo)...
2 arquivo(s) processado(s).
Registro salvo em 'hashes.txt'.
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> cat hashes.txt
a.txt:2ffeaa2e8fd6c3c83fc7e5918763c1b90595e1d38a6ccbacf8ab3fa41a0640dd
sub/b.txt:130eed0e858fb6a5f3412d8071c680f6037badd706bf7b424e40f7eef4d2b9f0
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> "conteudo A alterado" > dados/a.txt
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> "conteudo A alterado" > dados/a.txt
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> Remove-Item dados/sub/b.txt
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> "arquivo novo" > dados/c.txt
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação>

```

Figura 2 — Alteração, remoção e criação de arquivos na pasta 'dados'.

2.1.3 Teste — execução com a flag --verificar

O script foi executado novamente, agora com a flag --verificar, gerando o relatório de status:

```

PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> python3 verificador_integridade.py dados --verificar
Calculando hashes SHA-256 de todos os arquivos em 'dados' (recursivo)...
2 arquivo(s) processado(s).
=====
RELATÓRIO DE VERIFICAÇÃO DE INTEGRIDADE
=====

[+] Arquivos novos (1):
+ c.txt

[-] Arquivos removidos (1):
- sub/b.txt

[!] Arquivos modificados (1):
! a.txt

[=] Arquivos inalterados (0):

=====
Total de arquivos analisados: 3
=====
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação>

```

Figura 3 — Relatório de verificação de integridade, classificando corretamente os arquivos novos, removidos, modificados e inalterados.

Resultado: o script identificou corretamente c.txt como arquivo novo, sub/b.txt como removido e a.txt como modificado, validando a lógica de comparação implementada manualmente.

3. Item 2.2 — quebra_sem_salt.py

Script que testa hashes SHA-256 de senhas (sem salt) contra um dicionário de senhas comuns. Para cada senha do dicionário, o hash é calculado uma única vez (pré-computação), formando uma tabela reversa hash → senha, que é então consultada para cada hash-alvo fornecido pelo cliente.

3.1 Teste

Foi criado um dicionário de senhas comuns (senhas_comuns.txt) e um arquivo com dois hashes de teste: um correspondente à senha 'senha123' (presente no dicionário) e outro a uma senha fictícia ausente do dicionário.

```

PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> "123456" > senhas_comuns.txt
>> "password" >> senhas_comuns.txt
>> "senha123" >> senhas_comuns.txt
>> "admin" >> senhas_comuns.txt
>> "query" >> senhas_comuns.txt
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> python3 -c "import hashlib; print(hashlib.sha256('senha123').hexdigest()); print(hashlib.sha256('naoexiste0XZ').hexdigest());" > hashes_sem_salt.txt
55a5e67820794d16999d888f8a070094635470695d1a95bc7196b4eecd251:senha123
a4c1ef301bc9f02218c1ff40b07450f456a5f6a5312b6d72d3c60f9beeb321b7999:nao_ENCONTRADA

[RESUMO] 1/2 hash(es) quebrado(s) com o dicionário fornecido.
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação>

```

Figura 4 — Execução do quebra_sem_salt.py: a primeira senha foi encontrada, a segunda foi corretamente marcada como NAO_ENCONTRADA.

3.2 Resposta à pergunta do relatório

Por que esse método se torna inviável caso os hashes possuam salt?

Sem salt, o hash de uma senha é sempre o mesmo valor, independentemente de onde ou quando ela foi cadastrada. Isso permite construir uma única tabela pré-computada (hash → senha) a partir do dicionário, reaproveitável para qualquer hash-alvo do cliente. Com salt, cada senha é concatenada a um valor aleatório antes do cálculo do hash, de modo que a mesma senha gera hashes completamente diferentes de um registro para outro. Isso invalida o reaproveitamento de uma tabela única: seria necessário recalculá-lo o hash de cada senha do dicionário para cada salt individual, multiplicando o custo do ataque pelo número de salts distintos existentes.

O que mudaria exatamente na implementação?

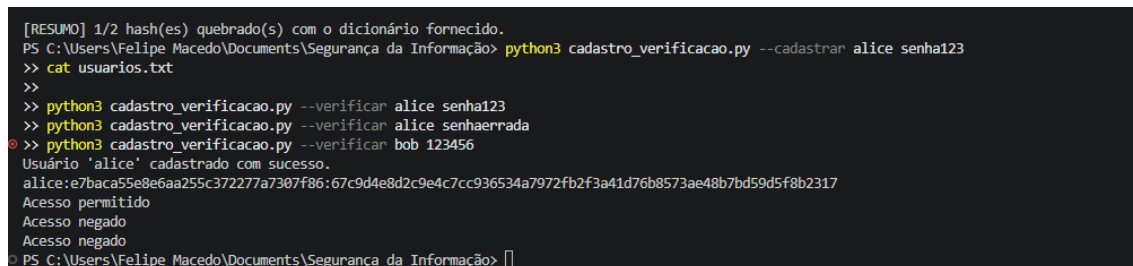
Em vez de calcular `hashlib.sha256(senha)` para montar a tabela do dicionário, seria necessário ler o salt correspondente a cada hash-alvo (formato `salt_hex:hash_hex`) e calcular `hashlib.sha256(salt + senha)` para cada candidato do dicionário, considerando aquele salt específico. Isso é exatamente o que foi implementado no item 2.4 (`quebra_com_salt.py`), com a otimização adicional de agrupar hashes que compartilham o mesmo salt para evitar recomputações desnecessárias.

4. Item 2.3 — `cadastro_verificacao.py`

Script que simula um módulo de cadastro e autenticação de usuários com salt individual por usuário. No cadastro, um salt aleatório de 16 bytes é gerado com `os.urandom`, concatenado à senha e o hash SHA-256 resultante é salvo junto ao salt no arquivo `usuarios.txt` (`usuario:salt_hex:hash_hex`). Na verificação, o salt armazenado é recombinado com a senha digitada para recalculá-lo e compará-lo com o hash.

4.1 Teste

Foi cadastrado o usuário 'alice' e testados três cenários de verificação: senha correta, senha incorreta e usuário inexistente ('bob').



```
[RESUMO] 1/2 hash(es) quebrado(s) com o dicionário fornecido.
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação> python3 cadastro_verificacao.py --cadastrear alice senha123
>> cat usuarios.txt
>>
>> python3 cadastro_verificacao.py --verificar alice senha123
>> python3 cadastro_verificacao.py --verificar alice senhaerrada
>> python3 cadastro_verificacao.py --verificar bob 123456
Usuário 'alice' cadastrado com sucesso.
alice:e7baca55e8e6aa255c372277a7307f86:67c9d4e8d2c9e4c7cc936534a7972fb2f3a41d76b8573ae48b7bd59d5f8b2317
Acesso permitido
Acesso negado
Acesso negado
PS C:\Users\Felipe Macedo\Documents\Segurança da Informação>
```

Figura 5 — Cadastro do usuário, conteúdo de `usuarios.txt` (sem senha em texto claro) e os três cenários de verificação de acesso.

Resultado: o cadastro foi concluído com sucesso, o arquivo `usuarios.txt` armazenou apenas o salt e o hash (nunca a senha em texto claro), e a verificação retornou 'Acesso permitido' para a senha correta e 'Acesso negado' tanto para a senha incorreta quanto para o usuário inexistente.

5. Item 2.4 — `quebra_com_salt.py`

Script que realiza um ataque de dicionário contra hashes com salt, no formato `salt_hex:hash_hex`. Diferente do item 2.2, aqui não é possível reutilizar uma única tabela pré-computada para todos os hashes, pois cada salt pode gerar um hash diferente para a mesma senha.

5.1 Desafio técnico (pré-computação otimizada)

Foi implementado o agrupamento dos hashes-alvo por salt: quando o mesmo salt aparece em múltiplos hashes do arquivo de entrada, o hash de cada senha do dicionário para aquele salt específico é

calculado apenas uma vez, e a tabela resultante é reaproveitada para todos os hashes daquele grupo em vez de recalculá-lo inteiro para cada linha do arquivo.

5.2 Teste

Foi gerado um arquivo de teste com três hashes: dois compartilhando o mesmo salt (senhas 'senha123' e 'qwerty') e um terceiro com um salt diferente e uma senha fora do dicionário.

```
PS C:\Users\Felipe Macedo\Documents\Seguranca da Informacao & "C:\Users\Felipe Macedo\AppData\Local\Programs\Python\Python311\python.exe" "C:\Users\Felipe Macedo\Documents\Seguranca da Informacao\gerar_teste_salt.py"
Arquivo hashes_com_salt.txt gerado.
PS C:\Users\Felipe Macedo\Documents\Seguranca da Informacao cat hashes_com_salt.txt
9c393236f3439ada5cb6cf1710ccf6f1:718277d45801bf22b944ac6f31faf9247d12c1616f1cab65c4c8f6762ba82bba
9c393236f3439ada5cb6cf1710ccf6f1:3942cc107a0b262fc39a0689f582a7d337118126e9e8f00b615f7e9e2602916f
153aa9a55ed398fee392e26b65815fb:bbc2148081a4c9f7ed72a8867c04728ba5b3cd8b8d70d41b1bfa545cb385a2
PS C:\Users\Felipe Macedo\Documents\Seguranca da Informacao pythons quebra_com_salt.py hashes_com_salt.txt senhas_comuns.txt
[INFO] 3 hash(es) alvo, agrupados em 2 salt(s) distintos(s).
9c393236f3439ada5cb6cf1710ccf6f1:718277d45801bf22b944ac6f31faf9247d12c1616f1cab65c4c8f6762ba82bba -> senha123
9c393236f3439ada5cb6cf1710ccf6f1:3942cc107a0b262fc39a0689f582a7d337118126e9e8f00b615f7e9e2602916f -> qwerty
153aa9a55ed398fee392e26b65815fb:bbc2148081a4c9f7ed72a8867c04728ba5b3cd8b8d70d41b1bfa545cb385a2 -> NAO_ENCONTRADA
[RESULT] 2/3 hash(es) quebrado(s).
PS C:\Users\Felipe Macedo\Documents\Seguranca da Informacao
```

Figura 6 — O script identificou 2 salts distintos entre os 3 hashes, reaproveitando a pré-computação para o salt repetido, e quebrou corretamente 2 dos 3 hashes.

Resultado: a mensagem 'agrupados em 2 salt(s) distinto(s)' confirma que o salt repetido foi tratado como um único grupo, validando a otimização de pré-computação exigida no desafio técnico (bônus).

6. Conclusão

Os quatro scripts foram implementados manualmente, respeitando integralmente as restrições impostas (uso de hashlib restrito ao cálculo unitário, proibição de bibliotecas prontas de quebra de senha, e autoria própria de toda a lógica de comparação, geração de salt e busca em dicionário). Os testes práticos, executados em ambiente Windows via PowerShell, confirmaram o funcionamento correto de todas as funcionalidades exigidas, incluindo o tratamento de arquivos grandes por leitura em blocos, a detecção de senhas fracas com e sem salt, o cadastro seguro de usuários e a otimização de pré-computação para salts reutilizados.

6.1 Resumo dos testes realizados

Script	Resultado do Teste
verificador_integridade.py	Classificou corretamente arquivos novos, removidos, modificados e inalterados.
quebra_sem_salt.py	Encontrou a senha correta no dicionário e marcou a ausente como NAO_ENCONTRADA.
cadastro_verificacao.py	Cadastrou o usuário, nunca armazenou senha em texto claro, validou acesso correto e negou acesso indevido.
quebra_com_salt.py	Agrupou corretamente hashes por salt reutilizado, quebrando 2 de 3 hashes com pré-computação otimizada.